

Young Innovators · Class 15

School Helpdesk Chatbot

Build a bot that answers questions about our school

Python

Keyword matching

Responsible AI

Did you know?

One of the first chatbots, ELIZA, was built in the 1960s using simple keyword rules, like ours!

HelpBot

When does school start?

School starts at 8:00.

Is there a robotics club?

Yes! Tue and Thu, Lab 2.



Let's think...

The same question, 50 times a day



Busy office

"When does school finish?" "When is the robotics club?" "What is the uniform on Friday?"

The school office answers the **same questions** again and again.
Could a program help?

Put up a poster?

Hire more staff?

Build a chatbot!

Today: you write a **Python chatbot** that answers school questions, then compare it with a modern AI chatbot.

HelpBot

When does school start?

School starts at 8:00.

Is there a robotics club?

Yes! Tue and Thu, Lab 2.





Learning aims

Today I will learn to...



1

Explain the difference between **rule-based** and **AI** chatbots

2

Plan questions, **keywords** and answers

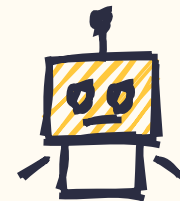
3

Code a chatbot in **Python** with a loop and a **dictionary**

4

**Test** it with real users and improve it

5

Use AI chatbots **responsibly** and check their answers**Keywords**chatbot · keyword · string · list ·
dictionary · loop · rule-based · AI

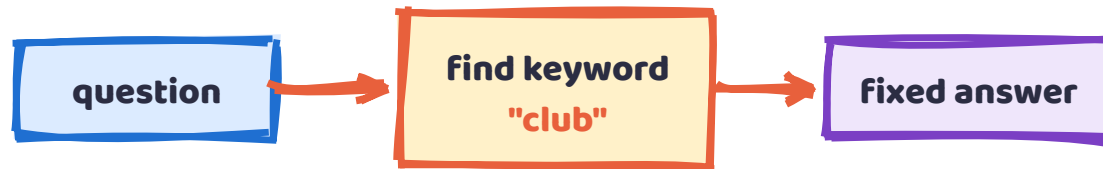


Big idea

Two kinds of chatbot



Rule-based (today)



The programmer writes every rule. **Predictable**, but only knows what you taught it.

Which is better?

It depends! For exact school timings, you want the answer to be **right every time**. A rule-based bot is great for that.



AI / large language model



Learns patterns from billions of words and **writes new text**. Flexible, but can sound sure while being **wrong**.

Did you know?

Many company helpdesks combine both: rules for important facts, AI for friendly chatting.





Real world

Chatbots are everywhere



Banks

Check your balance or block a lost card



Airlines

Change a booking or check a flight time



Universities

Answer questions about courses and admissions



Voice assistants

Answer spoken questions on phones and speakers



Have you ever talked to a chatbot? Did it understand you? What annoyed you?

Design lesson

The best bots quickly pass you to a human when they cannot help.





Plan first

Questions, keywords, answers



Question	Keywords	Answer
When does school start / finish?	time, start, finish	[real school timings]
When is robotics club?	robotics, club	[day, time, room]
What is the uniform?	uniform, wear	[real uniform rules]
Where is the library?	library, books	[location]
When are exams?	exam, test	[check the school calendar]

Collect real questions

Ask 5 students and a teacher:
what do people ask the office most?



Facts only!

Get answers from the school office or website. A chatbot that guesses is worse than none.



Python toolbox

The 5 tools our bot needs

```
name = input("What is your name? ") # ask the user
print("Hello", name)                # show text

q = "When Does School START?"
q = q.lower()                        # "when does school start?"
print("start" in q)                  # True

while True:                          # repeat forever...
    q = input("You: ")
    if q == "bye":
        break                        # ...until we break out
```

Tool	Job
input()	ask a question
print()	show an answer
.lower()	ignore CAPITALS
in	is a word inside?
while / break	repeat until done

Why lower()?

"Club", "CLUB" and "club" are different to Python.
lower() makes them all the same.



Code it: version 1

if, elif, else



```
print("HelpBot: Hi! Ask me about the school. Type bye to stop.")

while True:
    q = input("You: ").lower()
    if "bye" in q:
        print("HelpBot: Goodbye!")
        break
    elif "start" in q or "time" in q:
        print("HelpBot: School starts at 8:00.")      # use REAL info
    elif "club" in q:
        print("HelpBot: Robotics Club is on Tuesday.")
    else:
        print("HelpBot: Sorry, I don't know that yet.")
```

Read it like a story

Ask a question.

Bye? Stop.

Mentions time? Give the time.

Mentions club? Club answer.

Anything else? Say sorry.



Problem!

10 questions = 10 elif blocks. It gets long fast.
Version 2 fixes this.



Code it: version 2

A dictionary of answers

```
answers = {
    "time": "School starts at 8:00 and finishes at 2:30.",
    "club": "Robotics & AI Club: Tuesday and Thursday.",
    "uniform": "Full uniform Monday to Thursday.",
    "library": "The library is on the first floor.",
}

print("HelpBot: Hi! Ask me about the school. Type bye to stop.")
while True:
    q = input("You: ").lower()
    if "bye" in q:
        print("HelpBot: Goodbye!")
        break
    found = False
    for key in answers:
        if key in q:
            print("HelpBot:", answers[key])
            found = True
            break
    if not found:
        print("HelpBot: Sorry, I don't know that. Please ask the office.")
```

Dictionary

Pairs of **key : value**.

Key = the keyword.

Value = the answer.

Add a new answer with **one line**!

Example answers

These timings are EXAMPLES. Replace them with the real ones before anyone uses your bot!



Class game

Human chatbot



How to play

1. One student is the **bot** and may only answer using the rule cards on the desk.
2. Each card says: **if you hear [keyword], say [answer]**.
3. The class asks questions. Try to **break** the bot!
4. Record every question the bot could not answer.

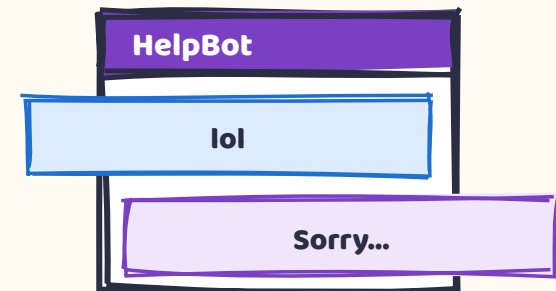
Then discuss

Which questions broke the bot?
Spelling? Two keywords at once?
Questions with no keyword?



Big idea

A rule-based bot is only as smart as its rules. Users always surprise you!





Get smarter

Learn from the questions you missed



```
unknown = []          # questions we could not answer

if not found:
    print("HelpBot: Sorry, I don't know that yet.")
    unknown.append(q)   # remember it!

# when the user types bye:
print("Questions to add next time:", unknown)
```

list

`unknown = []` makes an empty list.
`.append(q)` adds an item to the end.

This is how real bots improve

Teams read the questions their bot **failed** on, then add new keywords and answers. Humans are still in charge of what the bot says.

Privacy!

Questions can contain names or personal details. Don't save them in a shared file without permission.





Level 2



Synonyms, greetings and politeness

```
answers = {  
    ("time", "start", "finish", "timing"): "School starts at 8:00 ...",  
    ("club", "robotics", "after school"): "Robotics & AI Club ...",  
}  
for keys in answers:  
    for k in keys:  
        if k in q: ...
```

Order matters

Check more specific keywords first. "exam time" should give the exam answer, not the school time!

Synonyms

People say **timing**, **start**, or **finish**. Use a **tuple** of keywords for each answer.

Greetings

If the user says hi, hello or salam, greet them back by name.

Typos

Add common misspellings like **library** as extra keywords.



Test like an engineer

Test with real users



User typed

Bot answered

Helpful?

what time do we finish

yes / no

WHEN IS ROBOTICS

yes / no

where can i find books

yes / no

is there an exam soon

yes / no

hi!

yes / no

what's for lunch

yes / no

Debug checklist

No answer for an obvious question?

Add the keyword.

Wrong answer? Two keywords clashed:
reorder.

Let a friend type freely, without telling them the keywords. That is how real users behave!





Compare

Our bot vs a modern AI chatbot



	Our rule-based bot	AI chatbot
Knows OUR school's real timings	✓ if we typed them	✗ not unless told
Understands any wording	✗ keywords only	✓ very flexible
Same answer every time	✓	✗ can vary
Can invent wrong facts	✗ never	⚠ yes, sometimes
Explains how it decided	✓ read the code	✗ hard to tell

Teacher demo

The teacher asks an AI chatbot:
**“When does Pak-Turk Maarif
 Chak Shahzad finish today?”**
 Does it know? Does it admit it
 doesn't?

Big word

When an AI confidently makes up facts,
 people call it a **HALLUCINATION**.





Using AI responsibly

Smart users check the answers



Check facts: confirm important answers with a teacher, a book or an official website.



Keep private: never type your full name, address, passwords or photos into a chatbot.



Follow the rules: many AI apps have age limits. Follow them and your school's policy.



Be honest: if AI helped with your homework, say so. Learning is the goal.



Think first: use AI to help you think, not to do all the thinking.



Remember

An AI can sound very confident and still be wrong.





Debate



Should a bot pretend to be human?

Side A: tell people it is a bot

Users trust it the right amount. They know to check important answers. It is honest.

Side B: let it seem human

Some people feel more comfortable. But is it fair if they share secrets thinking a person is listening?



Vote, then debate for 5 minutes. Should our HelpBot start every chat with "I am a computer program"?

**Good practice**

Many rules and guidelines say people should be told when they are talking to an AI.



Level up!

Chatbot challenges

Choose your challenge



Time-aware: use the datetime module so "Is school open now?" gives a correct live answer.



Menu: if the bot is unsure, show a numbered menu of topics to choose from.



Window app: use tkinter to give your bot a chat window with a text box and button.



Score it: count how many keywords match each answer and pick the best one.

Think like an engineer

Our bot fails if a word is misspelled badly. How could a computer tell that "robtics" probably means "robotics"?

Boss level

Look up `difflib.get_close_matches()` in Python. It finds similar words!





Show what you know

What did we learn?

- ☒ Rule-based bots follow keywords that people write
- ☒ AI chatbots generate text and can make mistakes
- ☒ input, lower, in, while and dictionaries build our bot
- ☒ Real users and missed questions make a bot better
- ☒ Check AI answers and keep personal data private

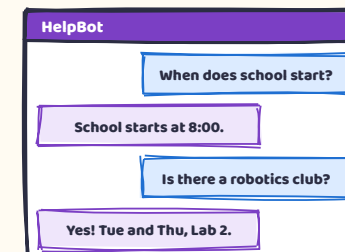


Exit ticket: name one thing our bot does better than an AI chatbot, and one thing it does worse.



At home

Ask a family member 3 questions they would want a school bot to answer. Add them to your bot!



Next class: 3D-Printed Plant Monitor Enclosure!

